

ArcNeuron: Reusing Depth for a Small Language Model That Can Spend More Compute on Reasoning

Lê Hùng Quang Minh
ArcatureLabs
lehungquangminh2011@gmail.com

August 2026

Abstract

ArcNeuron is an experimental decoder-only language model built around one simple question: can a small model reuse the same neural computation several times and obtain a useful reasoning benefit without turning the runtime into a collection of external rules? The model has a short prelude, a shared recurrent Transformer core, and a short coda. The core is applied repeatedly to an evolving hidden state while the original encoded context is injected at every iteration. Training remains ordinary next-token prediction on natural text. A later “tuning” stage simply continues the same objective at a lower learning rate on a small hand-guided corpus; it does not attach a classifier, symbolic solver, reward model, retrieval system, or task-specific adapter. This paper describes the architecture, the training discipline, and the experiments required before making claims about reasoning. The design is deliberately small enough to reproduce on a single GPU and simple enough that the full neural architecture fits in one Python file.

1 Why another small language model?

Large language models can hide weak experimental ideas behind scale. If a model has billions of parameters and has read a large fraction of the public web, it becomes difficult to tell whether a new behavior comes from the proposed mechanism, memorization, or simply more capacity. ArcNeuron starts from the opposite direction. The first model is intended to be small, the architecture is intentionally boring except for recurrent depth, and every proposed advantage has an ablation that can falsify it.

The project is motivated by three practical requirements. First, new concepts should not need thousands of nearly identical examples before they become usable. Second, harder problems should be allowed to consume more neural computation without allocating a different set of parameters for every extra step. Third, the final system must remain a normal language model: it reads token sequences and predicts token sequences. Python may schedule computation, but Python must not contain the knowledge or logic that makes an answer correct.

This distinction is important. ArcNeuron is not a symbolic reasoner with a neural front end. There is no dictionary mapping *chí* to *chân*, no rule saying that a mammal is an animal, and no branch that detects a mathematics question and calls a hidden solver. If the trained checkpoint is moved to another runtime that implements the same tensor operations, the model should produce the same distribution up to ordinary numerical differences.

2 Architecture

ArcNeuron is a causal decoder with three stages. Let $E(x)$ be token embeddings followed by a small prelude Transformer. The prelude output is called the context c and also initializes the recurrent state:

$$c = P(E(x)), \quad (1)$$

$$h_0 = c. \quad (2)$$

A shared recurrent core R is then applied d times. The core receives both the current state and the unchanged prelude context:

$$h_{i+1} = R([h_i; c]), \quad i = 0, \dots, d-1, \quad (3)$$

where $[h_i; c]$ denotes concatenation followed by a learned linear projection back to model width. A coda C and a tied language-model head produce logits:

$$z = W_{\text{emb}} \text{Norm}(C(h_d)). \quad (4)$$

The same parameters of R are used at every recurrence. Increasing d therefore increases compute but not parameter count. The idea is related to depth-recurrent language models such as Huginn [1], while the training and evaluation plan here is deliberately oriented toward small controlled experiments.

2.1 The Transformer block

Every block uses pre-normalization, grouped-query causal self-attention, rotary position embeddings, and a SwiGLU feed-forward network. In schematic form,

$$y = x + \text{Attention}(\text{RMSNorm}(x)), \quad (5)$$

$$x' = y + \text{SwiGLU}(\text{RMSNorm}(y)). \quad (6)$$

There is no custom attention rule. PyTorch’s scaled dot-product attention can dispatch to fused CUDA implementations on supported hardware. This is intentional: the first experiments should test the recurrence hypothesis rather than a new kernel.

The recurrent blocks use zero-initialized output projections on their residual branches. At initialization, the repeated block is therefore close to an identity transformation rather than an arbitrary destructive mapping. Recent controlled work on recurrent-depth Transformers reports that identity-friendly initialization can improve optimization of repeated computation [3].

2.2 Why inject the original context every time?

A recurrent hidden state has two jobs if it is left alone: preserve the input and transform it. These goals can conflict over many iterations. ArcNeuron keeps the prelude representation available at every recurrence so the state can change without being solely responsible for lossless storage of the original prompt. Geiping et al. found repeated input injection useful in their depth-recurrent scaling experiments [1]. ArcNeuron adopts the same broad principle with a simple concatenation-and-projection mixer.

2.3 What recurrence is not

Recurrence is not automatically reasoning. Probing work on depth-recurrent models found limited evidence that repeated hidden states form an interpretable latent chain of thought, and extra recurrence can produce only small gains on some tasks [2]. More recent controlled experiments also report an “overthinking” regime in which excessive recurrent depth hurts predictions [3]. ArcNeuron therefore treats depth as a hypothesis to test, not as a feature to advertise before measurement.

The key experiment is simple: hold weights fixed and evaluate the same examples at depths 1, 2, 4, and 8. If tasks requiring deeper composition improve up to a useful operating point, recurrence has earned its complexity. If they do not, the recurrent core should be removed.

3 Training from natural text

Base training uses one objective:

$$\mathcal{L}_{\text{LM}} = - \sum_t \log p_{\theta}(x_{t+1} \mid x_{\leq t}). \quad (7)$$

The corpus is plain text. There are no required Q:, A:, Reasoning:, or hidden label fields. This choice avoids teaching a small model that reasoning is a formatting convention. Explanations, counterexamples, uncertainty, correction, mathematics, code, and causal arguments appear as ordinary language.

Natural text alone does not remove the need for data design. A corpus containing only short factual statements mostly rewards short-range continuation. ArcNeuron’s intended corpus contains prose in which conclusions depend on earlier statements: explanations that separate correlation from implication, examples where insufficient information is the correct conclusion, arguments that revise an initial guess, and descriptions of a new concept followed by consequences that were not stated verbatim.

The distinction is between adding information and adding practice. Five sentences may contain only five facts about a new object, while many later passages can exercise different consequences of those facts without introducing new facts. This is similar in spirit to meta-learning work showing that neural networks can become much better at learning new meanings when the training distribution repeatedly requires few-shot learning and recombination rather than stable memorization [5, 6].

3.1 Context-dependent concepts

A useful training stressor is to reuse an invented surface form with unrelated meanings in different contexts. In one passage, *velan* might be an animal; elsewhere it might be a material or a data structure. The model cannot safely encode one permanent meaning for the token. It must use surrounding text to infer which representation is useful now.

This does not mean ordinary words should be randomly relabeled in the real corpus. The purpose is to include enough controlled text to reward contextual concept induction. Minnow demonstrates that repeated meta-training on new-word learning can substantially improve few-shot word learning [6]; MLC shows that systematic compositional behavior can be induced in standard neural architectures through the task distribution rather than symbolic machinery [5].

4 Variable recurrent depth

During base training, a complete microbatch uses one recurrent depth sampled from a small range such as $\{1, 2, 3, 4\}$. Using one depth for the whole microbatch keeps accelerator work aligned. More importantly, varying depth prevents the model from depending on one fixed number of transformations before prediction.

The first experiments do not need a learned halting network. A fixed user-selected depth is easier to understand and easier to benchmark. The model can later gain a stopping mechanism only if experiments show a clear benefit. This follows a general project rule: a mechanism is added after a measured failure, not before one is observed.

5 Tuning: continued training, not a second model

ArcNeuron uses the word *tuning* for a small, explicit “hand-guided” continuation stage. The base checkpoint is loaded, all of its parameters remain trainable, and next-token prediction continues at a lower learning rate on a smaller corpus. No adapter is attached and no alternate objective is introduced.

The tuning corpus demonstrates response behavior that is difficult to guarantee from broad pretraining alone: answer the central question early, explain enough reasoning to make a conclusion checkable, avoid inventing unsupported details, distinguish a property of an individual from a class definition, and revise a mistaken intermediate calculation when the contradiction is visible. A small amount of base-text replay can be mixed in to reduce catastrophic forgetting.

This stage should not carry the world’s knowledge. If a fact is present only in tuning data, the separation between “learning” and “being shown how to respond” has failed. The intended role of tuning is behavioral shaping of an already useful base model.

6 Reasoning, explanations, and hidden computation

ArcNeuron does not require a visible chain of thought marker. A response may be short when the task is simple and longer when the user needs justification. The recurrent core supplies extra hidden computation per predicted token, while autoregressive generation supplies sequential computation across generated tokens. These are two different axes of compute.

Explicit reasoning remains useful as training text when it is natural and concise. Work such as Quiet-STaR shows that learning useful rationales can improve prediction and downstream reasoning even when the training signal begins from ordinary text [7]. At the same time, latent-only reasoning should not be assumed to replace explicit intermediate reasoning at scale. LOTUS reports that looped Transformers can close this gap when latent positions receive direct supervision from reasoning steps [4]. That approach is deliberately left for a later ArcNeuron revision because it requires a more specialized training pipeline than the first experiment needs.

Self-verification is treated similarly. Evidence from S²R suggests that teaching models to inspect and correct their own reasoning can produce substantial gains [8]. ArcNeuron R1 does not add a verifier network. Instead, the tuning corpus contains ordinary prose in which a proposed conclusion is checked against evidence and corrected when necessary. More elaborate reinforcement learning belongs after the basic language-model path has been measured.

7 Tokenizer and independence from Python logic

The reference implementation uses SentencePiece BPE with byte fallback. BPE is a compression boundary, not a semantic database. The tokenizer learns recurring byte sequences and stores no hand-authored mapping such as “mammal means animal.” Byte fallback keeps arbitrary UTF-8 text representable even when a character was absent from the tokenizer corpus.

The serialized tokenizer model is stored inside the neural checkpoint. A checkpoint therefore contains the architecture configuration, ArcNeuron weights, and the exact text-to-ID codec. Moving the checkpoint to another correct implementation does not require `train.py`, `tune.py`, or any Python-side reasoning rule. Those scripts are optimization and sampling machinery only.

8 Reference configuration

The initial Colab-scale configuration is intentionally small:

Component	Value
Tokenizer target vocabulary	8192
Context length	1024
Model width	512
Query heads	8
Key/value heads	2
SwiGLU width	1408
Prelude blocks	1
Core blocks	2 shared
Coda blocks	1
Training recurrence	1-4
Evaluation recurrence	1, 2, 4, 8

Depending on the tokenizer’s actual vocabulary size, this configuration is roughly in the mid-teens of millions of unique parameters. At recurrence depth four, the computation traverses one prelude block, two shared core blocks four times, and one coda block. Effective depth increases while unique core parameters remain unchanged.

9 Experiments that can disprove the design

The project needs controlled baselines before scale. At minimum, the same tokenizer, data, optimizer budget, and approximate parameter count should be used for a standard Transformer and ArcNeuron. The test set should include:

- paraphrases that change wording without changing meaning;
- novel concepts defined by only a few observations;
- compositions whose answer is not written verbatim in the context;
- counterfactual changes to one property while other properties remain valid;
- contradictions and unsupported conclusions;
- requests where insufficient information is the correct response;
- natural explanations rather than only binary labels;
- mathematics and code problems with independently checkable answers;
- depth sweeps with the same weights at recurrence 1, 2, 4, and 8.

A successful result is not merely lower language-model loss. The recurrent model should improve specifically on tasks that need composition, and the improvement should correlate with additional recurrent compute up to a measured optimum. If a plain Transformer matches it at the same compute and parameter budget, the recurrence has not justified itself.

10 Implementation discipline

The reference repository intentionally keeps the model in one file. `arcneuron.py` contains only the neural architecture. It does not know how files are read, how loss is calculated, how a checkpoint is saved, how tokens are sampled, or what a question means. `train.py` performs base next-token training; `tune.py` continues the same training at lower learning rate; `generate.py` performs generic autoregressive sampling; `tokenizer.py` owns the reversible text codec.

This separation is not cosmetic. It is an audit rule. If removing a Python helper changes what the trained neurons know about cats, arithmetic, or program structure, then knowledge leaked outside the model. Runtime code is allowed to decide how many recurrent passes to execute and how to sample logits. It is not allowed to decide which answer is correct.

11 Limitations and next steps

ArcNeuron is a research proposal and reference implementation, not a claim of state-of-the-art reasoning. Several failure modes are plausible. Recurrent states may converge too quickly and gain little from extra depth. The core may overthink at high recurrence. Small language-model training may not create useful concept induction without a much better corpus. Tuning may improve answer style while damaging general language modeling. BPE may hide some low-level generalization effects that a byte model would expose, while a byte model would spend substantially more compute per piece of text.

The first milestone is therefore deliberately narrow: show that a small ArcNeuron checkpoint learns coherent language, can use a newly described concept in unseen prose, and gains measurable compositional accuracy when recurrence is increased within its useful range. Only after that should the project consider self-generated practice, reinforcement learning with verifiable rewards, learned stopping, latent-step supervision, longer context, or larger parameter counts.

12 Conclusion

ArcNeuron tries to make one architectural bet and keep everything around it ordinary. The bet is that reused neural depth can become a useful test-time reasoning resource when the training distribution rewards contextual learning, composition, explanation, and correction. The rest is standard language modeling: natural text, next-token cross entropy, a decoder-only Transformer, and autoregressive generation. This makes the idea cheap to reject if it fails and straightforward to scale if it works.

References

- [1] J. Geiping, S. McLeish, N. Jain, et al. *Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach*. arXiv:2502.05171, 2025. <https://arxiv.org/abs/2502.05171>
- [2] W. Lu, Y. Yang, K. Lee, Y. Li, and E. Liu. *Latent Chain-of-Thought? Decoding the Depth-Recurrent Transformer*. arXiv:2507.02199, 2025. <https://arxiv.org/abs/2507.02199>
- [3] H. Kohli, S. Parthasarathy, H. Sun, and Y. Yao. *Loop, Think, & Generalize: Implicit Reasoning in Recurrent-Depth Transformers*. arXiv:2604.07822, 2026. <https://arxiv.org/abs/2604.07822>
- [4] Y. Fan, A. Svete, and K. Lee. *Bridging the Gap Between Latent and Explicit Reasoning with Looped Transformers*. arXiv:2606.31779, 2026. <https://arxiv.org/abs/2606.31779>
- [5] B. M. Lake and M. Baroni. *Human-like systematic generalization through a meta-learning neural network*. *Nature* 623, 115–121, 2023. <https://www.nature.com/articles/s41586-023-06668-3>
- [6] W. Wang, G. Jiang, T. Linzen, and B. M. Lake. *Rapid Word Learning Through Meta In-Context Learning*. arXiv:2502.14791, 2025. <https://arxiv.org/abs/2502.14791>
- [7] E. Zelikman, G. Harik, Y. Shao, et al. *Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking*. arXiv:2403.09629, 2024. <https://arxiv.org/abs/2403.09629>
- [8] R. Ma, P. Wang, C. Liu, et al. *S²R: Teaching LLMs to Self-verify and Self-correct via Reinforcement Learning*. arXiv:2502.12853, 2025. <https://arxiv.org/abs/2502.12853>